Student Name: Kyle Patrick T. Mendoza

Group: Alpha

Assigned Role: File Management Developer

What did I contribute to the project?

I built the file management module for MiniOS Alpha. I created a system that lets users create, view, edit, rename, and delete files and folders using shell commands. I also made sure everything stays organized in a clear directory structure and actually saves to disk so your files don't disappear when you close the app.

What features did I personally develop?

I developed the file management functions in `filesystem.py`, here are what's inside:

- Directory creation "mkdir" – Allows users to create new folders.
- File creation "create" – Creates empty files.
- File listing "dir" – Displays files and directories, including file sizes.
- File reading "read" – Displays the contents of a file.
- File writing "write" – Saves text into a file.
- File deletion "delete" – Deletes files and empty directories.
- File and directory renaming "rename" – Changes the names of existing files and folders.
- Directory navigation "cd" and "pwd" – Allows users to move between directories and view their current location.
- Data persistence – Uses a JSON file named "minios_disk.json" to save the simulated file system so that its contents remain available after restarting the program.
- Error handling – Prevents invalid names, duplicate files, deletion of non-empty directories, and other invalid file operations.

What problems did I encounter?

One of the challenges was building a simulated file system that felt real without actually touching the user's real hard drive. I had to figure out how to handle path navigation, make sure users couldn't create duplicates or use weird invalid characters in file names, and keep all the data safely saved and reloaded between sessions.

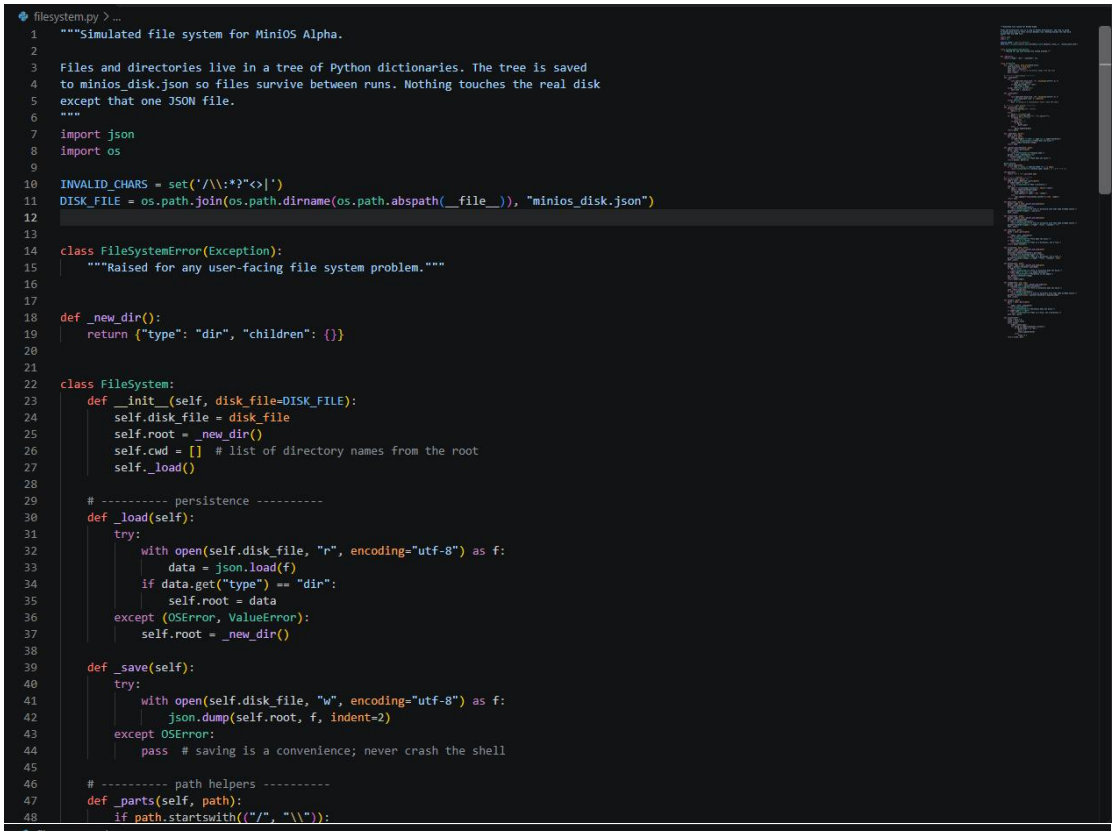
How did I solve those problems?

I built the file system using nested Python dictionaries to store the file tree. I wrote helper functions to parse paths, sanitize input, and prevent errors like broken paths or invalid file names. To give users clear feedback without crashing the app, I added custom exception handling, and I used Python's json module to save and reload the file tree between sessions.

What did I learn from this project?

I gained hands-on experience using nested data structures to model real-world hierarchies. It taught me how to handle path resolution, validate user input safely, and write clean, modular code that integrates smoothly into a larger system.

Evidence of my contribution:



filesystem.py > ...

```
"""Simulated file system for MiniOS Alpha.

Files and directories live in a tree of Python dictionaries. The tree is saved
to minios_disk.json so files survive between runs. Nothing touches the real disk
except that one JSON file.
"""
import json
import os

INVALID_CHARS = set('/\\:*?"<>|')
DISK_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "minios_disk.json")


class FileSystemError(Exception):
    """Raised for any user-facing file system problem."""

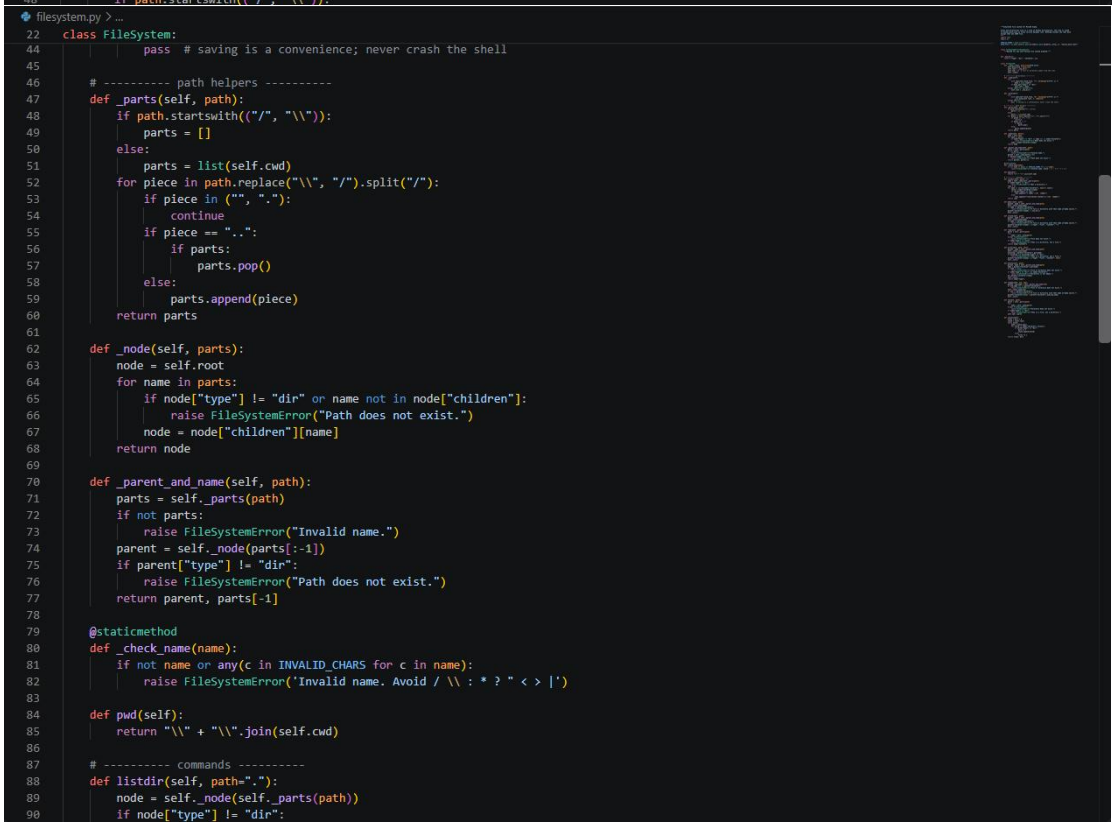

def _new_dir():
    return {"type": "dir", "children": {}}


class FileSystem:
    def __init__(self, disk_file=DISK_FILE):
        self.disk_file = disk_file
        self.root = _new_dir()
        self.cwd = []  # list of directory names from the root
        self._load()

    # ---------- persistence ----------
    def _load(self):
        try:
            with open(self.disk_file, "r", encoding="utf-8") as f:
                data = json.load(f)
            if data.get("type") == "dir":
                self.root = data
        except (OSError, ValueError):
            self.root = _new_dir()

    def _save(self):
        try:
            with open(self.disk_file, "w", encoding="utf-8") as f:
                json.dump(self.root, f, indent=2)
        except OSError:
            pass  # saving is a convenience; never crash the shell

    # ---------- path helpers ----------
    def _parts(self, path):
        if path.startswith(("/", "\\")):
```



filesystem.py > ...

```
class FileSystem:
            pass  # saving is a convenience; never crash the shell

    # ---------- path helpers ----------
    def _parts(self, path):
        if path.startswith(("/", "\\")):
            parts = []
        else:
            parts = list(self.cwd)
        for piece in path.replace("\\", "/").split("/"):
            if piece in ("", "."):
                continue
            if piece == "..":
                if parts:
                    parts.pop()
            else:
                parts.append(piece)
        return parts

    def _node(self, parts):
        node = self.root
        for name in parts:
            if node["type"] != "dir" or name not in node["children"]:
                raise FileSystemError("Path does not exist.")
            node = node["children"][name]
        return node

    def _parent_and_name(self, path):
        parts = self._parts(path)
        if not parts:
            raise FileSystemError("Invalid name.")
        parent = self._node(parts[:-1])
        if parent["type"] != "dir":
            raise FileSystemError("Path does not exist.")
        return parent, parts[-1]

    @staticmethod
    def _check_name(name):
        if not name or any(c in INVALID_CHARS for c in name):
            raise FileSystemError('Invalid name. Avoid / \\ : * ? " < > |')

    def pwd(self):
        return "\\" + "\\".join(self.cwd)

    # ---------- commands ----------
    def listdir(self, path="."):
        node = self._node(self._parts(path))
        if node["type"] != "dir":
```

filesystem.py > ...

```python
class FileSystem:

    # ---------- commands ----------
    def listdir(self, path="."):
        node = self._node(self._parts(path))
        if node["type"] != "dir":
            raise FileSystemError("Not a directory.")
        rows = []
        for name in sorted(node["children"], key=str.lower):
            child = node["children"][name]
            if child["type"] == "dir":
                rows.append(f"{'<DIR>':>10}  {name}")
            else:
                rows.append(f"{len(child['content']):>10}  {name}")
        return rows

    def mkdir(self, path):
        parent, name = self._parent_and_name(path)
        self._check_name(name)
        if name in parent["children"]:
            raise FileSystemError("A file or directory with that name already exists.")
        parent["children"][name] = _new_dir()
        self._save()

    def create(self, path):
        parent, name = self._parent_and_name(path)
        self._check_name(name)
        if name in parent["children"]:
            raise FileSystemError("A file or directory with that name already exists.")
        parent["children"][name] = {"type": "file", "content": ""}
        self._save()

    def read(self, path):
        parts = self._parts(path)
        try:
            node = self._node(parts)
        except FileSystemError:
            raise FileSystemError("File does not exist.")
        if node["type"] != "file":
            raise FileSystemError("That is a directory, not a file.")
        return node["content"]

    def write(self, path, text):
        parent, name = self._parent_and_name(path)
        self._check_name(name)
        existing = parent["children"].get(name)
        if existing and existing["type"] == "dir":
            raise FileSystemError("That is a directory, not a file.")
```

filesystem.py > ...

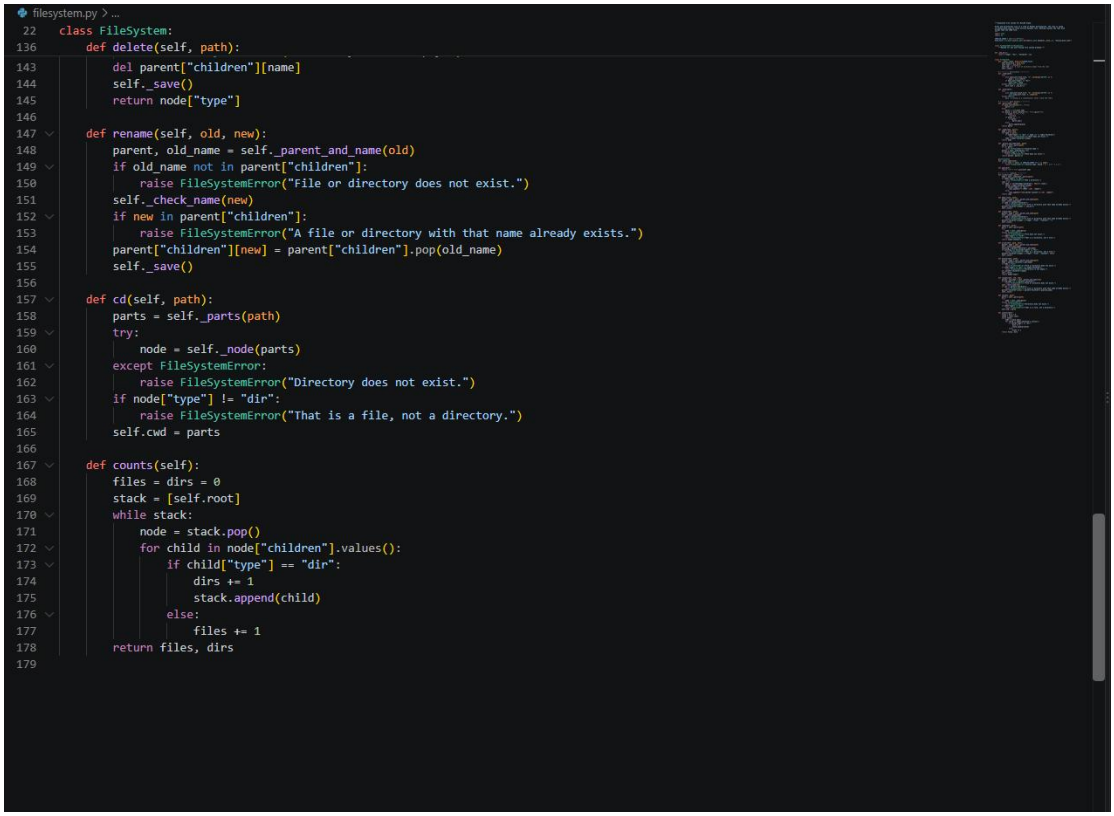
```python
class FileSystem:
    def write(self, path, text):
        existing = parent["children"].get(name)
        if existing and existing["type"] == "dir":
            raise FileSystemError("That is a directory, not a file.")
        parent["children"][name] = {"type": "file", "content": text}
        self._save()

    def delete(self, path):
        parent, name = self._parent_and_name(path)
        node = parent["children"].get(name)
        if node is None:
            raise FileSystemError("File or directory does not exist.")
        if node["type"] == "dir" and node["children"]:
            raise FileSystemError("Directory is not empty.")
        del parent["children"][name]
        self._save()
        return node["type"]

    def rename(self, old, new):
        parent, old_name = self._parent_and_name(old)
        if old_name not in parent["children"]:
            raise FileSystemError("File or directory does not exist.")
        self._check_name(new)
        if new in parent["children"]:
            raise FileSystemError("A file or directory with that name already exists.")
        parent["children"][new] = parent["children"].pop(old_name)
        self._save()

    def cd(self, path):
        parts = self._parts(path)
        try:
            node = self._node(parts)
        except FileSystemError:
            raise FileSystemError("Directory does not exist.")
        if node["type"] != "dir":
            raise FileSystemError("That is a file, not a directory.")
        self.cwd = parts

    def counts(self):
        files = dirs = 0
        stack = [self.root]
        while stack:
            node = stack.pop()
            for child in node["children"].values():
                if child["type"] == "dir":
                    dirs += 1
```


filesystem.py > ...
class FileSystem:
def delete(self, path):
del parent["children"][name]
self._save()
return node["type"]
def rename(self, old, new):
parent, old_name = self._parent_and_name(old)
if old_name not in parent["children"]:
raise FileSystemError("File or directory does not exist.")
self._check_name(new)
if new in parent["children"]:
raise FileSystemError("A file or directory with that name already exists.")
parent["children"][new] = parent["children"].pop(old_name)
self._save()
def cd(self, path):
parts = self._parts(path)
try:
node = self._node(parts)
except FileSystemError:
raise FileSystemError("Directory does not exist.")
if node["type"] != "dir":
raise FileSystemError("That is a file, not a directory.")
self.cwd = parts
def counts(self):
files = dirs = 0
stack = [self.root]
while stack:
node = stack.pop()
for child in node["children"].values():
if child["type"] == "dir":
dirs += 1
stack.append(child)
else:
files += 1
return files, dirs